

Centralized Identity Provider with Keycloak

Standards-based single sign-on and group-driven identity for Zero Trust applications

Project Implementation Report

Aparna J
Amity University Noida

1. Project Description

System Overview

This project deploys a self-hosted Identity Provider using Keycloak, an open-source IAM platform. In a Zero Trust framework, access decisions depend on one core requirement: verifying who the user is and what groups they belong to. While commercial SaaS platforms like Entra ID or Okta handle this in cloud environments, Keycloak delivers the same capability on infrastructure under your direct control.

The deployment runs Keycloak within a Docker container, establishes an isolated identity domain (realm), and configures role-based user groups. Downstream applications register as OpenID Connect (OIDC) clients, while custom protocol mappers inject group memberships directly into issued token claims. This yields a fully operational IdP that issues cryptographically signed JWTs carrying verified identity and role data—providing identity-aware proxies with the exact claims needed to enforce access policy.

Problem Statement

Managing user credentials independently across scattered applications creates severe operational friction. Users face constant password fatigue, administrators lose the ability to revoke access centrally, and authorization policies become fragmented across services.

This decentralized approach directly conflicts with Zero Trust, which requires continuous, centralized verification. Deploying a single authoritative IdP resolves these vulnerabilities: users authenticate once, and downstream systems receive standardized tokens they can validate statelessly.

Core Technology Stack

- **Identity Provider:** Keycloak, deployed in containerized mode for streamlined setup and evaluation.
- **Protocols:** OpenID Connect (OIDC) over OAuth 2.0, with native support for SAML 2.0.
- **Token Format:** Digitally signed JSON Web Tokens (JWT) carrying user identity and group assertions.
- **Deployment Runtime:** Docker containerization, ensuring rapid and reproducible deployments.

Key System Capabilities

- **Single Sign-On (SSO):** Users authenticate once to access all federated downstream workloads.
- **Realm Isolation:** Maintains logical identity boundaries per tenant, separating application directories from core system administration.
- **Group-Driven Claims:** Embeds group roles directly into token payloads to enable downstream, role-based authorization.
- **Standards-Based Federation:** Integrates OIDC and SAML-compliant applications natively, eliminating custom integration code.

Target Users

- **IAM & Infrastructure Engineers:** Operates the IdP platform and structures the organizational group model.

- **Application Developers:** Connects applications to a centralized login gateway instead of building bespoke authentication logic.
- **Security Operations:** Centralizes account revocation, audit logging, and security policy enforcement at the identity layer.

2. Application Scenarios and Use Cases

2.1 Centralized Authentication Across Applications

Organizations operating multiple internal applications often struggle to deliver a unified login experience. Keycloak addresses this by serving as the central Identity Provider. Instead of managing local logins, each application delegates authentication by redirecting users directly to Keycloak, trusting the cryptographically signed tokens returned upon successful login. This streamlines the experience for users—who maintain a single set of credentials—while giving administrators a single console to manage accounts and access rights.

2.2 Role-Driven Authorization Decisions

Granular access control requires more than basic authentication. By organizing users into functional groups (such as engineers or admins) and mapping these memberships into token claims, Keycloak enables downstream applications to enforce role-based access policies dynamically. This model mirrors how modern enterprise access brokers, such as Zscaler, consume identity assertions from providers like Entra ID to evaluate per-application permissions.

2.3 Standards-Based Identity Federation

Keycloak natively supports standard protocols like OpenID Connect and SAML, allowing compliant applications to federate seamlessly without custom integration code. Each realm exposes a standard discovery endpoint where connected applications automatically retrieve authentication endpoints and public signing keys. This standardized setup makes the Identity Provider instantly reusable across the entire enterprise ecosystem.

3. Technical Architecture Overview

High-Level Architecture

Keycloak anchors this architecture as the central identity authority. When a browser-based user accesses a protected resource, the request routes to a Keycloak realm for authentication. Keycloak validates the user's credentials, enforces mandatory security policies—such as Multi-Factor Authentication (MFA) enrollment—and issues a cryptographically signed JSON Web Token (JWT). Downstream applications registered as confidential OpenID Connect (OIDC) clients trust these tokens, while a custom protocol mapper injects group memberships directly into the token payload. Consequently, every issued token functions as a portable, self-contained assertion of identity and role.

- **Keycloak Server:** The central Identity Provider (IdP) operating on port 8080, hosting all realms and operational configurations.
- **Realm (zerotrust):** An isolated identity domain that serves as a single tenant boundary, decoupled from administration.
- **Groups & Users:** Configured engineers and admins groups with assigned users to drive downstream access decisions.
- **OIDC Client:** A confidential downstream application authenticated via a client secret.
- **Group Membership Mapper:** A protocol extension that writes user group memberships into the JWT under a groups claim.

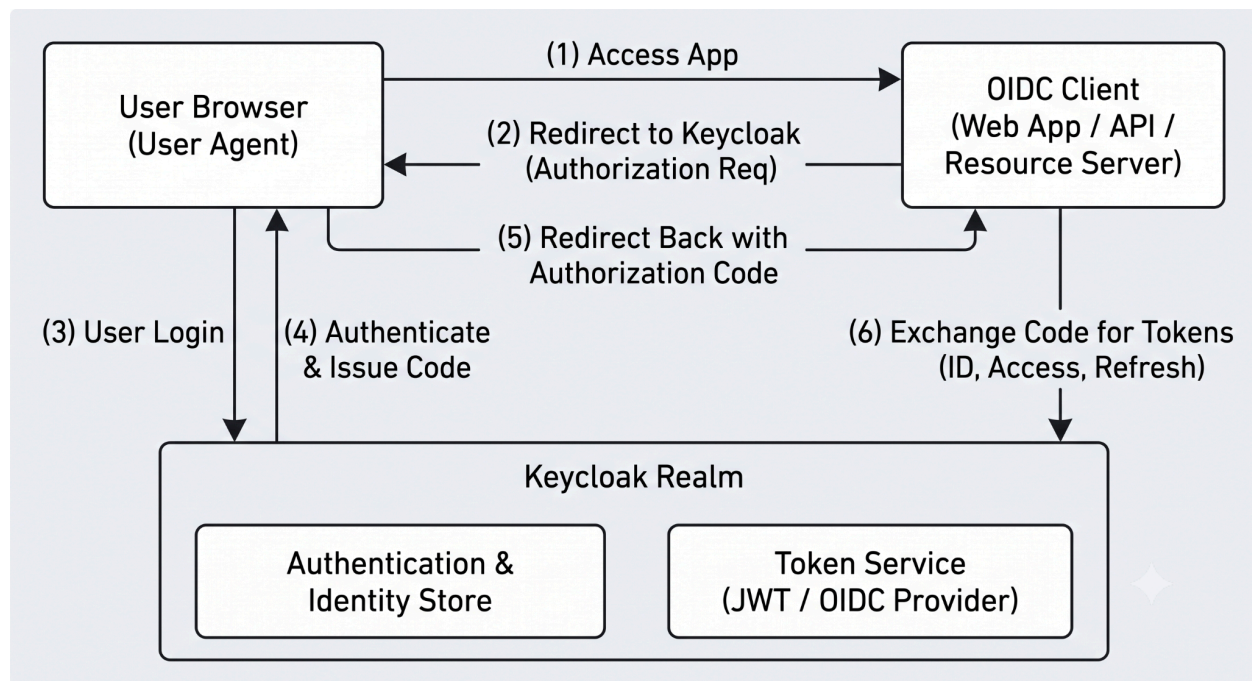


Fig 0 High-level identity architecture: browser, Keycloak realm, OIDC client (insert diagram)

Token Issuance Flow

1. **Access Request:** The user attempts to access a protected application and is redirected by the OIDC client to Keycloak.
2. **Credential Verification:** Keycloak presents the realm login interface and validates the user's credentials.

3. **Step-Up Challenge:** The IdP enforces any configured required actions, such as mandatory TOTP enrollment.
4. **Token Generation:** Keycloak issues a signed ID token and access token embedded with the user's group claims.
5. **Signature Validation:** The client application verifies the token's cryptographic signature against the realm's published public keys.
6. **Policy Enforcement:** The client reads the embedded group claim to render an automated access decision.

4. Prerequisites and Prior Knowledge

Software Requirements

- **Docker Engine:** Version 24.x or higher.
- **Web Browser:** Any modern browser for accessing the Keycloak administration console.
- **TOTP Authenticator:** Apps such as Google Authenticator or Authy for MFA setup.

Required Foundational Knowledge

- Core principles of OpenID Connect (OIDC) and OAuth 2.0.
- JWT architecture, claim payload structures, and cryptographic signature verification.
- Identity domain isolation concepts (realms and multi-tenancy).
- Basic operational commands for Docker containers.

Primary Project Objectives

- Deploy Keycloak as a centralized, standards-compliant OIDC and SAML identity provider.
- Establish an isolated realm configured with group models mapping directly to access tiers.
- Register a confidential OIDC client application to negotiate secure token exchanges.
- Embed user group memberships into issued tokens using custom protocol mappers.
- Enforce multi-factor authentication universally and validate the OIDC discovery endpoint.

Milestone 1: Requirement Analysis and Problem Definition

1.1 Problem Definition

Managing user identities across scattered, application-specific repositories severely limits centralized operational control. Without a single authoritative directory for credentials and group memberships, organizations struggle to enforce uniform authentication policies, revoke compromised access promptly, or provide consistent role signals for downstream authorization.

1.2 Proposed Solution

The solution deploys Keycloak as a unified IdP. By centralizing authentication, modeling access tiers through logical groups, and issuing standards-based tokens, downstream applications can delegate trust and validate user identities statelessly.

1.3 Functional Requirements

- **User & Directory Management:** Provision users, manage credentials, and assign group memberships centrally.
- **OIDC Client Registration:** Securely register confidential clients configured with exact redirect URIs and client secrets.
- **Claim Engineering:** Inject group memberships directly into ID and access token payloads.
- **MFA Enforcement:** Mandate a second authentication factor at login via central required actions.
- **Discovery Endpoint:** Expose standardized OIDC discovery metadata for integrated client services.

1.4 Non-Functional Requirements

- **Security:** Cryptographically sign all tokens, protect client secrets, and strictly isolate the master realm from application users.
- **Portability:** Containerize the IdP to ensure consistent operation across any Docker-compatible host.
- **Maintainability:** Support JSON export of realm configurations for version control and automated deployments.

1.5 Core Technology Stack

Component	Technology	Purpose
Identity Provider	Keycloak	Centralized authentication, SSO, and token issuance.
Auth Protocol	OpenID Connect (OIDC)	Federated identity layer built on OAuth 2.0.
Token	Signed JWT (RS256)	Portable container carrying identity and group claims.
Containerization	Docker Engine	Reproducible, isolated environment execution.

Milestone 2: Deploying the Keycloak Server

2.1 Launching the Container

Start Keycloak in development mode using the Docker CLI:

```
docker run -d --name keycloak -p 8080:8080 \
  -e KEYCLOAK_ADMIN=admin -e KEYCLOAK_ADMIN_PASSWORD=admin \
  quay.io/keycloak/keycloak:latest start-dev
```

Implementation Note: Running start-dev bypasses strict HTTPS requirements, making it ideal for local testing without TLS setup overhead.

Troubleshooting Pitfall: Newer Keycloak releases replace the administrative environment variables with KC_BOOTSTRAP_ADMIN_USERNAME and KC_BOOTSTRAP_ADMIN_PASSWORD. If the container terminates unexpectedly during launch, update these variable names in your deployment script.

Security Advisory: Development mode disables TLS enforcement and applies permissive security defaults. It is intended strictly for sandbox environments. Production deployments must run in production mode (start) behind trusted TLS, utilize an externalized database, and enforce administrative account hardening.

2.2 Creating the Realm

1. Navigate to <http://localhost:8080>, log into the administrative console using admin, and prepare to create a new realm.
2. Click the realm selection dropdown in the upper-left corner (currently displaying Master).
3. Select Create Realm.
4. Enter zerotrust as the realm name and click Create.
5. Verify that zerotrust is active in the realm selection menu.

Architecture Best Practice: A realm represents an isolated identity domain equivalent to a distinct organizational tenant. Application users must never be placed inside the master realm; master exists exclusively for system-level Keycloak administration, and mixing the two expands the blast radius of an administrative compromise.

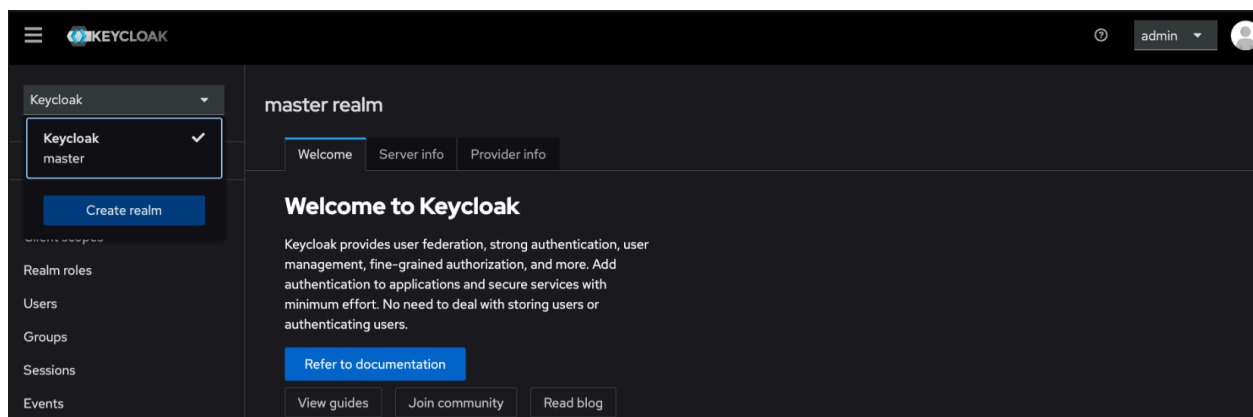


Fig 2.1 Creating the zerotrust realm (insert screenshot)

Milestone 3: User Directory & Group Architecture

3.1 Establishing Groups

Under Groups, create two functional access tiers: engineers and admins. Groups serve as the core driver for downstream access decisions, mirroring how modern network access brokers consume directory claims.

Group	Intended Access Tier
admins	Full administrative privileges across connected applications.
engineers	Standard operational access to engineering tools and services.

3.2 Provisioning Users & Assigning Membership

1. Navigate to Users, click Add user, enter a target username (e.g., alice), and save the record.
2. Open the Credentials tab, assign a password, and toggle Temporary to OFF.
3. Switch to the Groups tab, select Join Group, and assign the user to their designated group.
4. Repeat this process for a second user account (e.g., bob) in the alternate group.

Username	Group	Note
alice	engineers	Demonstration testing account
bob	admins	Demonstration testing account

Security Advisory: Demonstration accounts with static passwords belong strictly in isolated lab environments. Prior to deploying a realm, enforce rigid password policies, disable seed accounts, and mandate Multi-Factor Authentication.

Milestone 4: OIDC Client Integration & Custom Claim Engineering

4.1 Registering the Confidential Client

Downstream applications and identity-aware proxies require a registered client configuration to negotiate authentication workflows.

1. Navigate to Clients, click Create client, select OpenID Connect, assign a Client ID, and proceed.
2. Toggle Client authentication to ON to configure a confidential client.
3. Ensure Standard flow (Authorization Code Flow) is enabled.
4. Keep Direct access grants disabled unless explicitly required by legacy components.
5. Define Valid Redirect URIs using explicit callback URLs (e.g., `https://localhost/oauth2/callback`).
6. Save the configuration and copy the generated client secret from the Credentials tab.

Security Advisory: Always pin Valid Redirect URIs to exact endpoints. Using wildcards (*) on confidential clients exposes the system to open-redirect exploits and token leakage. Treat client secrets as high-value credentials: keep them out of source repositories and rotate them regularly.

4.2 Mapping Group Membership into Tokens

Keycloak omits group memberships from default token payloads. Attaching a custom protocol mapper ensures group assertions travel inside issued tokens.

1. Open the client's Client scopes tab and select the dedicated client scope.
2. Click Configure a new mapper and choose Group Membership.
3. Set both Name and Token Claim Name to groups.
4. Toggle Full group path to OFF so the claim outputs clean strings (e.g., admins instead of /admins).
5. Enable Add to ID token and Add to access token, then save.

Implementation Pitfall: Leaving Full group path enabled formats claims as /admins rather than admins. Downstream policy engines expecting simple string matches will silently fail, blocking legitimate user access. Ensure mapper outputs match policy formats precisely.

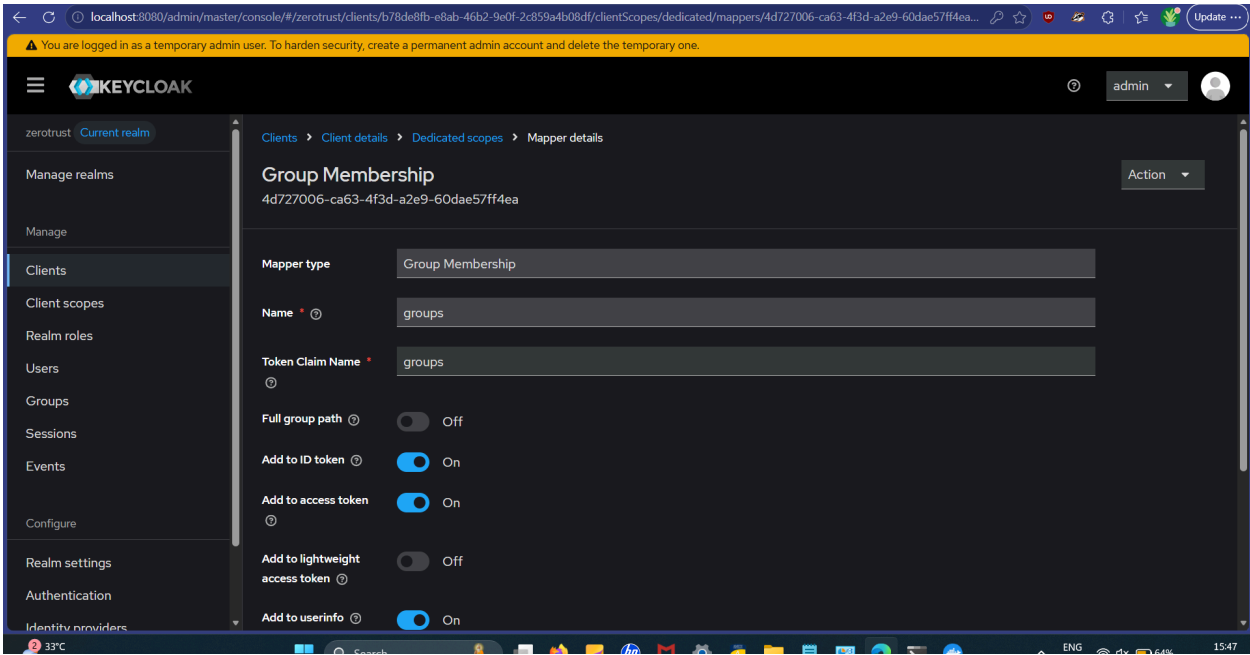


Fig 4.1 The Group Membership mapper producing the groups claim (insert screenshot)

Milestone 5: Multi-Factor Authentication (MFA)

5.1 Enforcing Mandatory OTP Enrollment

Enforcing MFA centrally at the IdP guarantees that every federated application inherits strong second-factor protections automatically.

1. Within the zerotrust realm, navigate to Authentication and select Required Actions.
2. Enable Configure OTP.
3. Set the action as Default Action to compel new users to enroll a TOTP application upon their next login.

Architecture Best Practice: Centralized MFA enforcement provides immediate security gains. A single rule at the IdP layer protects all current and future connected applications without requiring individual application updates.

Milestone 6: Testing & System Validation

6.1 Validating the OIDC Discovery Endpoint

Verify that the realm successfully publishes its OIDC metadata document. Connected applications query this endpoint to discover authentication paths and public key sets:

```
http://localhost:8080/realms/zerotrust/.well-known/openid-configuration
```

A valid response returns a structured JSON payload listing the authorization, token, and JWKS URIs.

6.2 Verification Testing Results

Test Case	Expected Result	Status
Discovery Metadata	Endpoint returns valid OIDC JSON containing realm keys and routes.	Pass
User Authentication (alice)	System authenticates credentials and triggers mandatory OTP enrollment.	Pass
Group Claim Generation	Issued JWT contains "groups": ["engineers"] in payload.	Pass
Claim Formatting	Claim outputs engineers cleanly rather than /engineers.	Pass
Confidential Exchange	Code-for-token exchange succeeds using valid client credentials.	Pass
Invalid Credentials	System blocks access attempt and issues no tokens.	Pass

7. Security Analysis and Threat Model

7.1 Protected Assets & Trust Boundaries

- **Protected Assets:** User credentials, realm signing keys, client secrets, and active tokens.
- **Identity Boundary:** The zerotrust realm, serving as the sole authority for authentication and role claims.
- **Administrative Boundary:** The master realm, strictly isolated from standard application traffic.

7.2 Threat Mitigation Matrix

Threat Description	Architectural Mitigation
Credential sprawl	A single authoritative IdP authenticates all integrated services.
Weak authentication	Centralized MFA enforcement prevents single-factor credential compromise.
Token forgery	Tokens are cryptographically signed; clients validate signatures against published JWKS keys.
Redirect Hijacking	Redirect URIs are explicitly pinned to exact callback endpoints.
Privilege Escalation	Application accounts are strictly restricted to tenant realms, isolating the master console.

7.3 Residual Risks & Production Remediation

Identified Sandbox Risk	Required Production Remediation
Development mode (no TLS)	Deploy Keycloak in production mode (start) behind valid TLS certificates.
Weak demonstration accounts	Enforce password complexity policies and remove seed user accounts.
Hardcoded secrets	Inject client secrets via environment variables or secret vaults.
In-memory database	Externalize data storage to a high-availability relational database.

8. Strategic Advantages, Limitations, and Future Enhancements

Key Advantages

- **Unified Identity Control:** A single pane of glass for user authentication, session revocation, and security auditing.
- **Standards Compliance:** Native OIDC and SAML 2.0 support eliminates custom integration code.
- **Role-Driven Authorization:** Token claims provide downstream proxies with the exact assertions needed for policy enforcement.
- **Operational Portability:** Containerized deployment allows rapid replication and exportable JSON configurations.

Architectural Limitations

- **Non-Production Runtime:** Development mode lacks TLS enforcement and relies on permissive defaults.
- **Ephemeral Data State:** In-memory storage does not persist data across container teardowns.
- **Single-Node Deployment:** The current lab configuration lacks high-availability clustering and load balancing.

Future Enhancements

- Transition Keycloak to production mode behind an enterprise load balancer with an externalized database.
- Incorporate WebAuthn and FIDO2 hardware keys alongside TOTP for phishing-resistant MFA.
- Federate external identity providers (such as corporate Active Directory or cloud IdPs) for broader identity synchronization.
- Manage realm configurations as code using GitOps pipelines and JSON exports.
- Deploy fine-grained authorization services to enforce resource-level policies.

9. Conclusion

This project successfully establishes the foundational identity layer for a Zero Trust architecture. By deploying Keycloak as a centralized Identity Provider, the architecture delivers cryptographically signed tokens containing verified identity assertions and role-based group claims. With isolated realms, structured user directories, confidential client integrations, custom claim mappers, and universal MFA enforced, downstream services gain a reliable, standards-compliant source of trust.

Resume summary: Deployed Keycloak as a centralized OIDC and SAML Identity Provider, configuring multi-tenant realms, group-based token claims, universal MFA, and federated SSO for Zero Trust architectures.

Appendix

A.1 Command Reference

Command	Description
<code>docker logs keycloak</code>	View Keycloak startup and runtime logs
<code>docker restart keycloak</code>	Restart the Keycloak container
<code>curl .../.well-known/openid-configuration</code>	Fetch the realm discovery document

A.2 Troubleshooting Guide

Symptom	Likely Cause	Resolution
Container exits on start	Wrong admin env var names	Use <code>KC_BOOTSTRAP_ADMIN_USERNAME</code> and <code>_PASSWORD</code>
Downstream policy denies valid user	Group claim is /admins not admins	Set Full group path OFF on the mapper
Discovery endpoint 404	Wrong realm name in the URL	Confirm the realm is zerotrust and re-check the path
<code>redirect_uri</code> error at login	Redirect URI mismatch	Pin the exact callback in the client settings
Users gone after restart	Dev mode in-memory database	Persist data or re-import the realm export

A.3 Source Code

GitHub repository: add the project repository link here.